

Week 4: Order and Order Details Management System – ArtHub

Scenario

ArtHub is an online art marketplace where customers can browse and purchase different artworks. The system needs to maintain information about customer orders and the individual artworks included in each order.

The **Orders** table stores the overall information about an order, such as the customer who placed the order, order date, total amount, and order status. The **Order Details** table stores the details of the artworks purchased in each order, including the artwork, quantity, and price.

The Order and Order Details Management System helps ArtHub maintain order information accurately and establish relationships between customers, orders, and artworks.

1. Design Orders and Order Details Tables

Orders Table

The **Orders** table is created to store the main information related to customer orders.

The table contains the following attributes:

- **Order_ID** – Unique identifier for each order.
- **Customer_ID** – Identifies the customer who placed the order.
- **Order_Date** – Stores the date on which the order was placed.
- **Total_Amount** – Stores the total amount of the order.
- **Order_Status** – Stores the current status of the order.
- **OrderDetails Table**

The **OrderDetails** table stores information about the individual artworks included in an order.

The table contains:

- **Order_Detail_ID** – Unique identifier for each order detail.
- **Order_ID** – Identifies the corresponding order.
- **Artwork_ID** – Identifies the artwork purchased.
- **Quantity** – Stores the quantity of artwork ordered.
- **Price** – Stores the price of the artwork.

```
SQL> CREATE TABLE Orders (Order_ID NUMBER, Customer_ID NUMBER, Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));
Table created.

SQL> CREATE TABLE OrderDetails (Order_Detail_ID NUMBER, Order_ID NUMBER, Artwork_ID NUMBER, Quantity NUMBER, Price NUMBER(10,2));
Table created.
```

2. Establish Relationships

Customer and Orders

One customer can place multiple orders, while each order belongs to one customer.

Relationship: Customer → Orders = One-to-Many

The Customer_ID in the Orders table is used to establish this relationship.

Orders and OrderDetails

One order can contain multiple order details, while each order detail belongs to one order.

Relationship: Orders → OrderDetails = One-to-Many

The Order_ID in OrderDetails connects each order detail with its corresponding order.

ArtProduct and OrderDetails

The OrderDetails table is connected with the ArtProduct table through Artwork_ID. This identifies the artwork purchased in each order.

This relationship connects the artwork catalogue with customer purchase details.

3. Apply Constraints

Primary key constraints were applied to uniquely identify records in both tables.

- **Orders:** Order_ID is the Primary Key.
- **OrderDetails:** Order_Detail_ID is the Primary Key.

Foreign key constraints were established to maintain relationships between the tables.

- Customer_ID in Orders references the Customer table.
- Order_ID in OrderDetails references the Orders table.
- Artwork_ID in OrderDetails references the ArtProduct table.

NOT NULL constraints were applied to important fields to ensure that required information is not left empty.

CHECK constraints were also applied to maintain valid data:

- Order quantity must be greater than zero.
- Artwork price must be greater than zero.
- Total order amount must be greater than or equal to zero.

These constraints help maintain the accuracy, consistency, and integrity of the ArtHub database.

PRIMARY KEY CONSTRAINT:

```
SQL> ALTER TABLE Orders ADD CONSTRAINT pk_orders PRIMARY KEY (Order_ID);  
Table altered.
```

```
SQL> ALTER TABLE OrderDetails ADD CONSTRAINT pk_orderdetails PRIMARY KEY (Order_Detail_ID);  
Table altered.
```

FOREIGN KEY CONSTRAINT:

```
SQL> ALTER TABLE Orders ADD CONSTRAINT fk_orders_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID);  
Table altered.
```

```
SQL> ALTER TABLE OrderDetails ADD CONSTRAINT fk_orderdetails_orders FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);  
Table altered.
```

```
SQL> ALTER TABLE OrderDetails ADD CONSTRAINT fk_orderdetails_artwork FOREIGN KEY (Artwork_ID) REFERENCES ArtProduct(Artwork_ID);  
Table altered.
```

NOT NULL CONSTRAINT:

```
SQL> ALTER TABLE Orders MODIFY Customer_ID NUMBER NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Orders MODIFY Order_Date DATE NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Orders MODIFY Total_Amount NUMBER(10,2) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Orders MODIFY Order_Status VARCHAR2(20) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE OrderDetails MODIFY Artwork_ID VARCHAR2(10) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE OrderDetails MODIFY Quantity NUMBER NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE OrderDetails MODIFY Price NUMBER(10,2) NOT NULL;  
Table altered.
```

CHECK CONSTRAINTS:

```
SQL> ALTER TABLE OrderDetails ADD CONSTRAINT chk_orderdetails_quantity CHECK (Quantity > 0);
Table altered.

SQL> ALTER TABLE OrderDetails ADD CONSTRAINT chk_orderdetails_price CHECK (Price > 0);
Table altered.

SQL> ALTER TABLE Orders ADD CONSTRAINT chk_orders_total CHECK (Total_Amount >= 0);
Table altered.
```

DESCRIBE THE STRUCTURE OF THE TABLES:

```
SQL> desc orders;
Name                                     Null?      Type
-----
ORDER_ID                               NOT NULL   VARCHAR2(10)
CUSTOMER_ID                            NOT NULL   NUMBER
ORDER_DATE                             NOT NULL   DATE
TOTAL_AMOUNT                           NOT NULL   NUMBER(10,2)
ORDER_STATUS                           NOT NULL   VARCHAR2(20)

SQL> desc orderdetails;
Name                                     Null?      Type
-----
ORDER_DETAIL_ID                        NOT NULL   VARCHAR2(10)
ORDER_ID                               NOT NULL   VARCHAR2(10)
ARTWORK_ID                            NOT NULL   VARCHAR2(10)
QUANTITY                              NOT NULL   NUMBER
PRICE                                  NOT NULL   NUMBER(10,2)
```

4. Insert Values into Orders and OrderDetails:

Seven sample records were inserted into the **Orders** table. The orders were identified using values such as **O1 to O7**.

The inserted records contain customer IDs, order dates, total amounts, and different order statuses such as **Processing, Shipped, and Delivered**.

Seven corresponding records were inserted into the **OrderDetails** table using detail IDs such as **OD1 to OD7**. Each record contains an order ID, artwork ID, quantity, and price.

The inserted values provide sample order information for testing relationships, joins, reports, and CRUD operations.

```
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O1', 1, TO_DATE('20-09-2026', 'DD-MM-YYYY'), 5000, 'Delivered');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O2', 2, TO_DATE('21-09-2026', 'DD-MM-YYYY'), 5200, 'Shipped');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O3', 3, TO_DATE('21-09-2026', 'DD-MM-YYYY'), 1800, 'Delivered');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O4', 4, TO_DATE('22-09-2026', 'DD-MM-YYYY'), 2200, 'Processing');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O5', 5, TO_DATE('22-09-2026', 'DD-MM-YYYY'), 3500, 'Shipped');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O6', 6, TO_DATE('23-09-2026', 'DD-MM-YYYY'), 4000, 'Delivered');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status) VALUES ('O7', 7, TO_DATE('24-09-2026', 'DD-MM-YYYY'), 2800, 'Processing');
1 row created.
```

```

SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD1', '01', '101', 1, 5000);
1 row created.
SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD2', '02', '102', 1, 5200);
1 row created.
SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD3', '03', '103', 1, 1800);
1 row created.
SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD4', '04', '104', 1, 2200);
1 row created.
SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD5', '05', '105', 1, 3500);
1 row created.
SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD6', '06', '106', 1, 4000);
1 row created.
SQL> INSERT INTO OrderDetails (Order_Detail_ID, Order_ID, Artwork_ID, Quantity, Price) VALUES ('OD7', '07', '107', 1, 2800);
1 row created.

```

SELECT BOTH TABLES:

```

SQL> select*from orders;

```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	TOTAL_AMOUNT	ORDER_STATUS
01		1 20-SEP-26	5000	Delivered
02		2 21-SEP-26	5200	Shipped
03		3 21-SEP-26	1800	Delivered
04		4 22-SEP-26	2200	Processing
05		5 22-SEP-26	3500	Shipped
06		6 23-SEP-26	4000	Delivered
07		7 24-SEP-26	2800	Processing

```

7 rows selected.

SQL> select*from orderdetails;

```

ORDER_DETA	ORDER_ID	ARTWORK_ID	QUANTITY	PRICE
OD1	01	101	1	5000
OD2	02	102	1	5200
OD3	03	103	1	1800
OD4	04	104	1	2200
OD5	05	105	1	3500
OD6	06	106	1	4000
OD7	07	107	1	2800

```

7 rows selected.

```

5. Generate Customer Order History Reports

SQL queries were used to generate reports for analyzing customer orders and artwork purchases in the ArtHub online art marketplace.

Report 1: Customer Order History

The Customer Order History report displays the order information associated with each customer.

Display:

- Customer Name
- Order ID
- Order Date
- Total Amount

- Order Status

The report is generated by joining the **Customer** and **Orders** tables using Customer_ID. Additional order-detail information can also be displayed when required.

This report helps identify the orders placed by each customer along with the date, amount, and current status of the order.

```
SQL> SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date, o.Total_Amount, o.Order_Status FROM Customer c JOIN Orders o ON c.Customer_ID = o.Customer_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE
1	Dhoni	01	20-SEP-26
5000	Delivered		
2	Virat	02	21-SEP-26
5200	Shipped		
3	Ashwin	03	21-SEP-26
1800	Delivered		
4	SanjuSamson	04	22-SEP-26
2200	Processing		
5	Shreyas Iyer	05	22-SEP-26
3500	Shipped		
6	Hardhik Pandya	06	23-SEP-26
4000	Delivered		
7	Ruturaj Gaikwad	07	24-SEP-26
2800	Processing		

Report 2: Artwork-wise Order Report

The Artwork-wise Order Report provides information about the ordering frequency and quantity sold for each artwork.

Find:

- Number of times each artwork was ordered.
- Total quantity sold.

The report joins the **ArtProduct** and **OrderDetails** tables using Artwork_ID.

The COUNT() function is used to determine the number of times an artwork was ordered, while the SUM() function is used to calculate the total quantity sold.

This report helps Arthub analyze the ordering activity of individual artworks.

```
SQL> SELECT a.Artwork_ID, a.Artwork_Name, COUNT(o.Order_ID) AS Number_of_Times_Ordered, SUM(o.Quantity) AS Total_Quantity_Sold FROM ArtProduct a JOIN OrderDetails o ON a.Artwork_ID = o.Artwork_ID GROUP BY a.Artwork_ID, a.Artwork_Name;
```

ARTWORK_ID	ARTWORK_NAME	NUMBER_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
104	Charcoal Dreams	1	1
102	Colors of Nature	1	1
106	Future City	1	1
101	Sunset Over Hills		

Report 3: Customer Purchase Analysis

The Customer Purchase Analysis report is used to analyze the purchasing activity of customers.

Find:

- Customers with maximum orders.
- Total spending by each customer.
- Average order value.

The report joins the **Customer** and **Orders** tables using Customer_ID.

The COUNT() function is used to calculate the number of orders, SUM() is used to calculate total spending, and AVG() is used to calculate the average order value.

This report provides a summarized view of customer purchasing activity in the ArtHub system.

```
SQL> SELECT c.Customer_ID, c.Customer_Name, COUNT(o.Order_ID) AS Total_Orders, SUM(o.Total_Amount) AS Total_Spending, AVG(o.Total_Amount) AS Average_Order_Value FROM Customer c JOIN Orders o ON c.Customer_ID = o.Customer_ID GROUP BY c.Customer_ID, c.Customer_Name ORDER BY Total_Orders DESC;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
4	SanjuSamson	1	2200	2200
1	Dhoni	1	5000	5000
6	Hardhik Pandya	1	4000	4000
7	Ruturaj Gaikwad	1	2800	2800
5	Shreyas Iyer	1	3500	3500
2	Virat	1	5200	5200
3	Ashwin	1	1800	1800

7 rows selected.

6. CRUD Operations

CRUD operations were performed on the **OrderDetails** table to demonstrate basic data management.

Update Operation

An existing order-detail record was updated by changing its quantity. This demonstrates how existing order information can be modified when a customer changes the quantity of an artwork.

```
SQL> UPDATE OrderDetails SET Quantity = 2 WHERE Order_Detail_ID = 'OD7';
```

1 row updated.

```
SQL> SELECT * FROM OrderDetails WHERE Order_Detail_ID = 'OD7';
```

ORDER_DETA	ORDER_ID	ARTWORK_ID	QUANTITY	PRICE
OD7	07	107	2	2800

Delete Operation

An existing order-detail record was deleted from the table. This demonstrates how an order-detail record can be removed from the database when it is no longer required.

The table was checked after the operations to verify the changes.

```
SQL> DELETE FROM OrderDetails WHERE Order_Detail_ID = 'OD7';
```

```
1 row deleted.
```

```
SQL> SELECT * FROM OrderDetails;
```

ORDER_DETA	ORDER_ID	ARTWORK_ID	QUANTITY	PRICE
OD1	01	101	1	5000
OD2	02	102	1	5200
OD3	03	103	1	1800
OD4	04	104	1	2200
OD5	05	105	1	3500
OD6	06	106	1	4000

```
6 rows selected.
```

Conclusion

The **Order and Order Details Management System** was successfully implemented for the ArtHub online art marketplace. The Orders and OrderDetails tables were created with appropriate keys and constraints, and relationships were established between customers, orders, and artworks.

Sample order data was inserted and SQL joins were used to generate customer order history, artwork-wise order, and customer purchase analysis reports. CRUD operations were also performed on the order details.

The Week 4 implementation provides an organized database structure for storing, managing, and analyzing customer orders and artwork purchases in the ArtHub system.